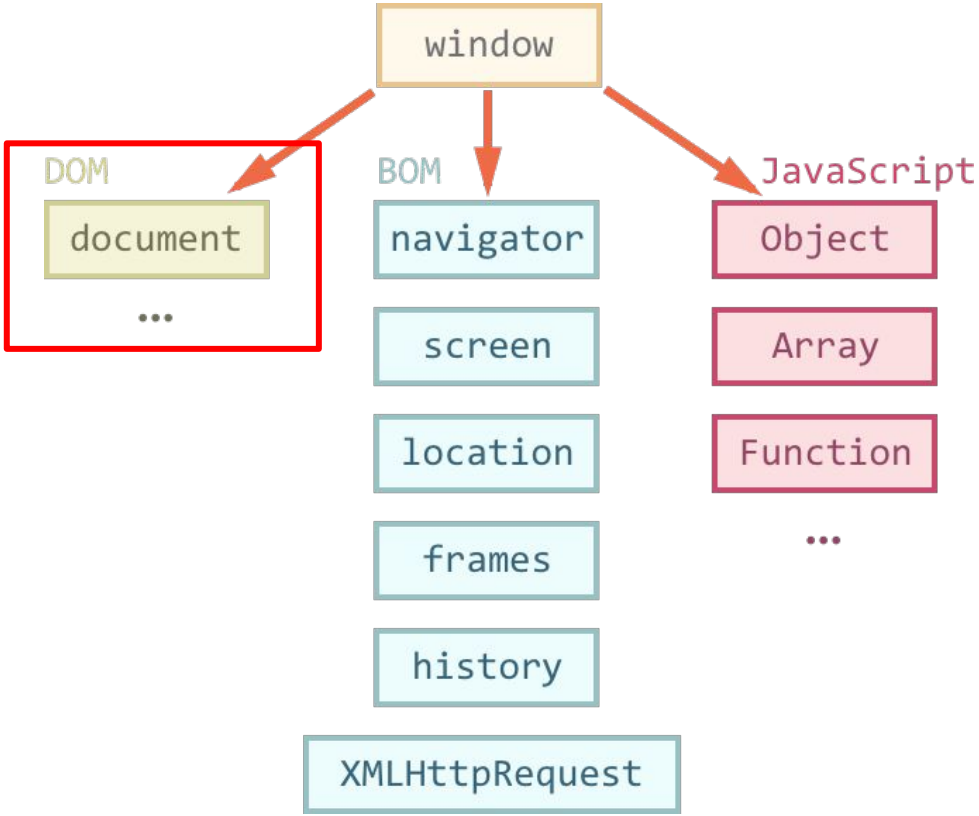
อัพเกรดเว็บด้วย JavaScript

Javascript

Javascript สามารถเข้าถึง **Elements** ทั้งหมดในหน้าเว็บที่สร้างมาจากภาษา **HTML** โดยมี **Browser** เป็นตัวประมวลผลคำสั่งต่างๆ ปัจจุบันนิยมใช้ Javascript ทำสิ่งเหล่านี้

- HTML document traversal
- Manipulation
- Event handling
- Animation
- Ajax

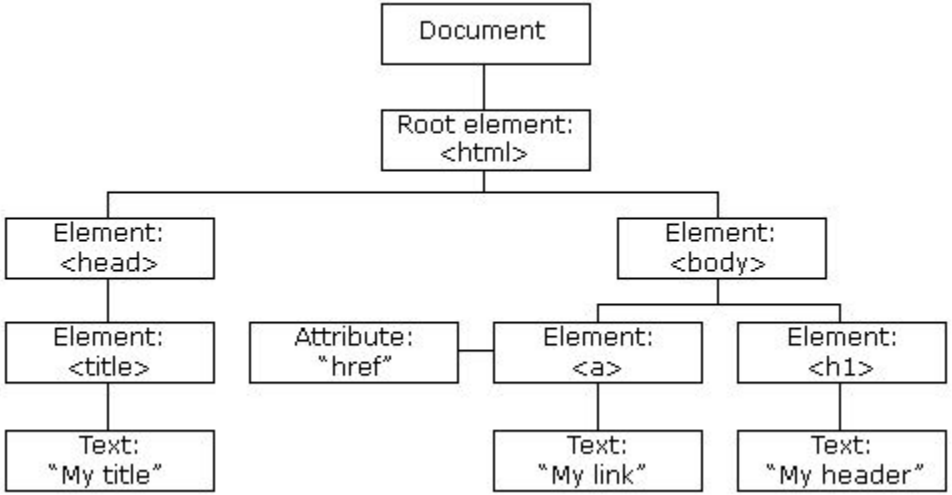
Browser environment, specs



DOM (Document Object Model)

The Document Object Model เป็นอินเทอร์เฟซและแพลตฟอร์มที่ช่วยให้โปรแกรมและสคริปต์สามารถเข้าถึงและอัปเดตเนื้อหาโครงสร้าง รูปแบบ การแสดงผลของเอกสารแบบไดนามิกได้

Javascript ถูกนำมาใช้เพื่อจัดการ DOM (Document Object Model)



document.body = <body>

สามารถเข้าถึงส่วน body ของหน้าเว็บ HTML ได้ผ่าน **document.body**

```
<html>

<head>
   <script>
       alert("From HEAD: " + document.body); // null, there's no <body> yet
   </script>
</head>

<body>
   <script>
       alert("From BODY: " + document.body); // HTMLBodyElement, now it exists
   </script>
</body>

</html>
```

Searching Elements

document.getElementById or just id

ถ้า **Element (ตัวแรก)** มี **id** สามารถเข้าถึง **Element** นี้ได้ผ่าน **document.getElementById** หรือ **id** ตรงๆ

```
<html lang="en">
<body>
   <div id="elem">
       <div id="elem-content">Element</div>
   </div>

   <script>
       let elem = document.getElementById( 'elem');
       elem.style.background =  'red';
       elem.style.width =  'auto';
       elem.style.height =  200;
   </script>
</body>
</html>
```

document.getElementsByTagName

หา **Elements (หลายตัว)** ด้วยชื่อ **Tag**

```
<html lang="en">
<body>
   <table id="table">
       <tr>
           <td>Your age:</td>
           <td>
               <label><input type="radio" name="age" value="young" checked> less than
18</label>
               <label><input type="radio" name="age" value="mature"> more than 18</label>
           </td>
       </tr>
   </table>
   <script>
       let inputs = document.getElementsByTagName ('input');
       for (let input of inputs) {
           alert(input.value + ': ' + input.checked);
       }
     alert(inputs[0].value + ': ' + inputs[0].checked);
   </script>
</body>
</html>
```

document.querySelectorAll

หา **Elements (หลายตัว)** ด้วย **Selector**

```
<html lang="en">
<body>
   <ul>
       <li>The</li>
       <li>test</li>
   </ul>
   <ul>
       <li>has</li>
       <li>passed</li>
   </ul>
   <script>
       let elements = document.querySelectorAll('ul > li:last-child');
       for (let elem of elements) {
           alert(elem.innerHTML); // "test", "passed"
       }
   </script>
</body>
</html>
```

document.querySelector

หา **Element (ตัวแรก)** ด้วย **Selector**

```
<html lang="en">
<body>
   <h1>Contents</h1>
   <div class="contents">
       <ul class="book">
           <li class="chapter">Chapter 1</li>
           <li class="chapter">Chapter 2</li>
       </ul>
   </div>
   <script>
       let chapter = document.querySelector('.chapter'); // LI

       alert(chapter.closest('.book').innerHTML); // UL
       alert(chapter.closest('.contents').innerHTML); // DIV

       alert(chapter.closest('h1')); // null (because h1 is not an ancestor)
   </script>
</body>
</html>
```

Live Collections

```
<html lang="en">
<body>
   <div>First div</div>
   <script>
      let divs = document.getElementsByTagName('div');
      alert(divs.length);      // 1
   </script>
   <div>Second div</div>
   <script>
       alert(divs.length);     // 2
   </script>
</body>
</html>
```

Static Collections

```
<html lang="en">
<body>
   <div>First div</div>
   <script>
      let divs = document.querySelectorAll('div');
      alert(divs.length);     // 1
   </script>
   <div>Second div</div>
   <script>
      alert(divs.length);     // 1
   </script>
</body>
</html>
```

Method	Searches by...	Can call on an element?	Live?
`getElementById`	`id`	-	-
`getElementsByName`	`name`	-	✔
`getElementsByTagName`	tag or `'*'`	✔	✔
`getElementsByClassName`	class	✔	✔
`querySelector`	CSS-selector	✔	-
`querySelectorAll`	CSS-selector	✔	-

Contents

elem.innerHTML

elem.innerHTML เข้าถึง HTML ภายใน Element เพื่อดึงค่าและแก้ไขได้ในรูปแบบ String

```
<html lang="en">

<body>
   <p>A paragraph</p>
   <div>A div</div>
   <script>
       alert(document.body.innerHTML); // read the current contents
       document.body.innerHTML = 'The new BODY!'; // replace it
   </script>
</body>

</html>
```

elem.outerHTML

elem.outerHTML เข้าถึง HTML ทั้งหมดของ Element เพื่อดึงค่าและแก้ไขค่าในรูปแบบ String

```
<html lang="en">

<body>
   <div>Hello, world!</div>
   <script>
       let div = document.querySelector('div');

       // replace div.outerHTML with <p>...</p>
       div.outerHTML = '<p>A new element!</p>'; // (*)

       // Wow! The div is still the same!
       alert(div.outerHTML); // <div>Hello, world!</div>
   </script>
</body>

</html>
```

elem.textContent

elem.textContent เข้าถึง Text ภายใน Element เพื่อดึงค่าและแก้ไขได้ในรูปแบบ String

```
<html lang="en">

<body>
   <div id="news">
       <h1>Headline!</h1>
       <p>Martians attack people!</p>
   </div>
   <script>
       // Headline! Martians attack people!
       alert(news.textContent);
   </script>
</body>

</html>
```

innerHTML Vs textContent

```
<html lang="en">

<body>
   <div id="elem1"></div>
   <div id="elem2"></div>
   <script>
       let name = prompt("What's your name?", "<b>Winnie-the-pooh!</b>");
       elem1.innerHTML = name;
       elem2.textContent = name;
   </script>
</body>

</html>
```

Winnie-the-pooh!

Winnie-the-pooh!

Attributes and Properties

HTML Attributes

ในภาษา HTML แล้ว Tag อาจมีหลาย Attributes เมื่อ Browser อ่าน HTML และสร้าง DOM รับรู้คุณลักษณะมาตรฐาน และสร้าง **DOM Properties** จาก HTML เหล่านั้น

id และ Attributes มาตรฐานอื่น ๆ จะถูกสร้างเป็น **DOM Properties** แต่ Attributes ไม่ได้เป็นมาตรฐาน จะไม่ถูกกำหนดเข้าไปใน DOM Properties

```
<html lang="en">
<body id="test" something="non-standard">
   <input id="input" type="text">
   <script>
       alert(document.body.id); // test
       // non-standard attribute does not yield a property
       alert(document.body.something); // undefined

       // input
       alert(input.type); // text
       alert(body.type); // undefined: DOM property not created, because it's non-standard
   </script>
</body>
</html>
```

Attributes สามารถเข้าถึงด้วย Methods เหล่านี้

- elem.**hasAttribute**(name) – checks for existence.
- elem.**getAttribute**(name) – gets the value.
- elem.**setAttribute**(name, value) – sets the value.
- elem.**removeAttribute**(name) – removes the attribute.

```
<html lang="en">
<body>
   <div id="elem" about="Elephant"></div>
   <script>
       alert(elem.getAttribute('About')); // (1) 'Elephant', reading
       elem.setAttribute('Test', 123); // (2), writing
       alert(elem.outerHTML); // (3), see it's there

       for (let attr of elem.attributes) { // (4) list all
           alert(`${attr.name} = ${attr.value}`);
       }
   </script>
</body>
</html>
```

Style Attribute เป็น String แต่ Style Property เป็น Object

```
<html lang="en">

<body>
   <div id="div" style="color:red;font-size:120%" >Hello</div>
   <script>
       // string
       alert(div.getAttribute('style')); // color:red;font-size:120%
       // object
       alert(div.style);      // [object CSSStyleDeclaration]
       alert(div.style.color);    // red
   </script>
</body>

</html>
```

href ใน DOM Property เป็น full URL

```
<html lang="en">

<body>
   <a id="a" href="#hello">link</a>
   <script>
       // attribute
       alert(a.getAttribute('href')); // #hello
       // property
       alert(a.href); // full URL in the form http://site.com/page#hello
   </script>
</body>

</html>
```

Classes and Styles

Classes

Classes ถูกใช้ในการกำหนดการแสดงผล **Elements** ใน **HTML** การใช้ Javascript ในการเข้าถึง **Classes** สามารถทำได้โดยใช้ **methods** เหล่านี้

elem.className

การเข้าถึงเพื่อดึงค่าและแก้ไข class ของ Element สามารถใช้ className ใน DOM Properties

```
<html lang="en">
<body class="main page">
   <script>
       alert(document.body.className);  // main page
   </script>
</body>
</html>
```

elem.classList

elem.classList จะเก็บ class ของ Element แยกออกมาเป็นแต่ละอัน ไว้ใน Array

```
<html lang="en">
<body class="main page">
   <script>
       for (let name of document.body.classList) {
           alert(name);  // main, and then page
       }
   </script>
</body>
</html>
```

elem.classList

elem.classList เป็น Property ที่ภายในมี methods สำหรับจัดการ class ของ Element อยู่ประกอบด้วย

- elem.classList.**add**/**remove**("class") – **adds** / **removes** the class.
- elem.classList.**toggle**("class") – if the class exists, then removes it, otherwise adds it.
- elem.classList.**contains**("class") – returns **true** / **false**, checks for the given class.

```
<html lang="en">
<body class="main page">
   <script>
       // add a class
       document.body.classList.add( 'article');
       alert(document.body.className);  // main page article
   </script>
</body>
</html>
```

Styles

elem.style

elem.style เป็น Property สำหรับจัดการ style ของ Element

ภายใน elem.style สามารถกำหนดค่า backgroundColor, Color, marginTop และอื่นๆ

```
<html lang="en">

<body>
   <script>
       document.body.style.backgroundColor = prompt('background color?', 'green');
   </script>
</body>

</html>
```

elem.style.cssText

elem.style.cssText ใช้เมื่อต้องการกำหนดคุณสมบัติของ style หลายแบบในคำสั่งเดียว

```
<html lang="en">

<body>
   <div id="div">Button</div>
   <script>
       // we can set special style flags like "important" here
       div.style.cssText = `color: red !important;
           background-color: yellow;
           width: 100px;
           text-align: center;
       `;
       alert(div.style.cssText);
   </script>
</body>

</html>
```

getComputedStyle

การดึงค่า **style** แต่ละแบบ ไม่สามารถดึงขึ้นมาได้โดยตรงผ่าน **elem.style** ต้องใช้ **method** ชื่อ **getComputedStyle** ช่วยก่อน

```
<html lang="en">
<head>
   <style>
       body {
           color: red;
           margin: 5px
       }
   </style>
</head>
<body>
   The red text
   <script>
       alert(document.body.style.color); // empty
       alert(document.body.style.marginTop); // empty
   </script>
</body>
</html>
```

getComputedStyle

```
<html lang="en">
<head>
   <style>
       body {
           color: red;
           margin: 5px
       }
   </style>
</head>
<body>
   <script>
       let computedStyle = getComputedStyle(document.body);
       // now we can read the margin and the color from it
       alert(computedStyle.marginTop); // 5px
       alert(computedStyle.color); // rgb(255, 0, 0)
   </script>
</body>
</html>
```

Modifying the document

Creating an element

การสร้าง DOM Nodes สามารถทำได้ผ่าน 2 methods

document.createElement

```
let div = document.createElement('div');
```

document.createTextNode

```
let textNode = document.createTextNode('Here I am');
```

Insertion Methods

การเพิ่ม Element เข้าไปใน document สามารถทำได้ผ่าน methods เหล่านี้

elem.appendChild

```html
<html lang="en">

<body>
   <ol id="list">
       <li>0</li>
       <li>1</li>
       <li>2</li>
   </ol>
   <script>
       let newLi = document.createElement('li');
       newLi.innerHTML = 'Hello, world!';
       list.appendChild(newLi);
   </script>
</body>

</html>
```

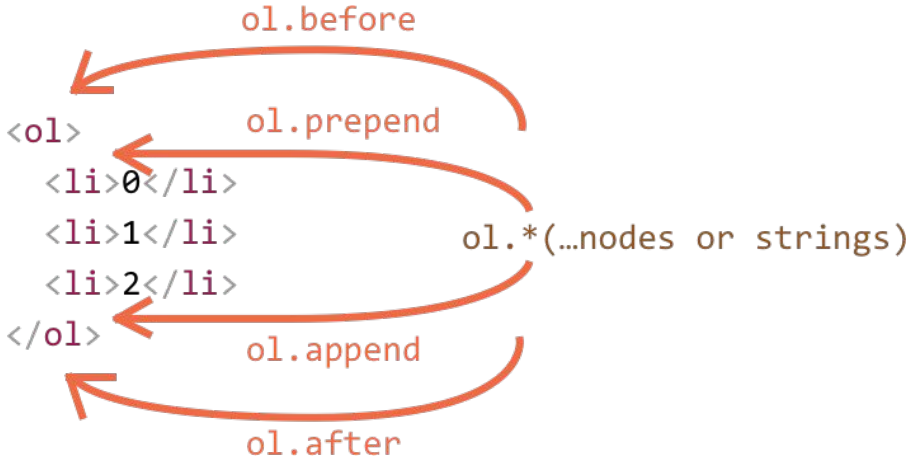
```html
<html lang="en">

<body>
   <ol id="list">
       <li>0</li>
       <li>1</li>
       <li>2</li>
       <li>Hello, world!</li>
   </ol>
</body>

</html>
```

append / prepend / before / after / replaceWith

- elem.**append**(...nodes or strings) – append nodes or strings at the end of node.
- elem.**prepend**(...nodes or strings) – insert nodes or strings into the beginning of node.
- elem.**before**(...nodes or strings) – insert nodes or strings before the node.
- elem.**after**(...nodes or strings) – insert nodes or strings after the node.
- elem.**replaceWith**(...nodes or strings) – replaces node with the given nodes or strings.



prepend / append

```html
<html lang="en">
<body>
   <ol id="ol">
       <li>0</li>
       <li>1</li>
       <li>2</li>
   </ol>

   <script>
       let prepend = document.createElement('li');
       prepend.innerHTML = 'prepend';
       ol.prepend(prepend);

       let append = document.createElement('li');
       append.innerHTML = 'append';
       ol.append(append);
   </script>
</body>
</html>
```

```html
<html lang="en">

<body>
     <ol id="list">
          <li>prepend</li>
          <li>0</li>
          <li>1</li>
          <li>2</li>
          <li>append</li>
   </ol>
</body>

</html>
```

before

```
<html lang="en">
<body>
   <div id="div"></div>
   <script>
       div.before('<p>Hello</p>',
          document.createElement('hr'));
   </script>
</body>
</html>
```

```
<html lang="en">
<body>
     <p>Hello</p>;
     <hr>
     <div id="div"></div>
</body>
</html>
```

after

```
<html lang="en">
<body>
   <div id="first">First</div>
   <div id="second">Second</div>
   <script>
       second.after(first);
   </script>
</body>
</html>
```

```
<html lang="en">
<body>
   <div id="second">Second</div>
   <div id="first">First</div>
</body>
</html>
```

cloneNode

การ clone DOM Node สามารถทำได้ผ่าน **cloneNode** ใช้

- **true** - clone of the element with all attributes and subelements.
- **false** - clone is made without child elements.

```
<html lang="en">
<body>
   <div class="alert" id="div">
       <strong>Hi there!</strong> You've read an important message.
   </div>
   <script>
       let div2 = div.cloneNode(true);
       div2.querySelector('strong').innerHTML = 'Bye there!'; // change the clone

       div.after(div2); // show the clone after the existing div
   </script>
</body>
</html>
```

remove / removeChild

ลบ DOM Nodes สามารถทำได้จาก methods เหล่านี้

- elem.**remove** ลบ Node
- parentElem.**removeChild** ลบ Child Node ของ parentElem

```
<html lang="en">
<head>
   <title>Document</title>
</head>
<body>
   <script>
       let div = document.createElement('div');
       div.className = "alert alert-success";
       div.innerHTML = "<strong>Hi there!</strong> You've read an important message.";

       document.body.append(div);
       setTimeout(() => div.remove(), 1000);
       // or setTimeout(() => document.body.removeChild(div), 1000);
   </script>
</body>
</html>
```

Events

Events

Events คือ การกระทำหรือสิ่งที่เกิดขึ้นกับ **HTML Elements** เมื่อนำ Javascript มาใช้กับเว็บไซต์ Javascript สามารถจัดการ ตอบสนองกับ Events ที่เกิดขึ้นเหล่านี้ได้

โดย **Events** สามารถเป็นสิ่งที่เกิดขึ้นจาก **Browser** หรือ **User** เป็นคนทำ ตัวอย่างเช่น

- เว็บไซต์โหลดและแสดงหน้าเสร็จสมบูรณ์
- กดปุ่มบนหน้าเว็บไซต์
- คลิ๊กเมาส์บนหน้าเว็บไซต์
- ลากเมาส์ไปวางบนรูปภาพ ปุ่ม หรือตำแหน่งใดตำแหน่งหนึ่งบนหน้าเว็บ
- การกดคีบอร์ด
- กรอกข้อมูลลงบน Form
- และอื่นๆ

Common Events

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

HTML Attribute

```
<html lang="en">
<body>
   <button onclick="alert(this.innerHTML)">Click me</button>
</body>
</html>
```

DOM Property

```
<html lang="en">
<body>
   <input id="elem" type="button" value="Click me">
   <input id="elem2" type="button" value="Say Thank" >
   <script>
       elem.onclick = function () {
           alert('Thank you');
       };

       function sayThanks() {
           alert('Thanks!');
       }
       elem2.onclick = sayThanks;
   </script>
</body>
</html>
```

window

```
<html lang="en">

<body>
   <script>
       window.onload = function () {
           alert('Page loaded');
           // image is loaded at this time
           alert(`Image size: ${img.offsetWidth}x${img.offsetHeight}`);
       };
   </script>
   <img id="img" src="https://en.js.cx/clipart/train.gif?speed=1&cache=0">
</body>

</html>
```

elem.addEventListener

```
<html lang="en">
<body>
   <input id="elem" type="button" value="Click me" />

   <script>
       function handler1() {
           alert('Thanks!');
       };

       function handler2() {
           alert('Thanks again!');
       }

       elem.onclick = () => alert("Hello");
       elem.addEventListener("click", handler1); // Thanks!
       elem.addEventListener("click", handler2); // Thanks again!
   </script>
</body>
</html>
```

Event Object

```
<html lang="en">
<body>
   <input type="button" value="Click me" id="elem">
   <script>
       elem.onclick = function (event) {
           // show event type, element and coordinates of the click
           alert(event.type + " at " + event.currentTarget);
           alert("Coordinates: " + event.clientX + ":" + event.clientY);
       };
   </script>
   <button id="elem">Click me</button>
   <script>
       elem.addEventListener('click', {
           handleEvent(event) {
               alert(event.type + " at " + event.currentTarget);
           }
       });
   </script>
</body>
</html>
```

Browser default action

Events หลายๆ ตัวถูกจัดการโดยอัตโนมัติด้วย Browser ตัวอย่างเช่น

- การคลิ๊กที่ Link บนหน้าเว็บเพื่อเปลี่ยนหน้า
- การคลิ๊กปุ่ม Submit บน Form
- และอื่นๆ

ถ้าไม่ต้องการให้ Browser เป็นตัวจัดการ Events เหล่านี้โดยอัตโนมัติ สามารถทำได้ 2 วิธีดังนี้

```
<html lang="en">
<body>
   <ul>
       <li>
           <a href="/" onclick="return false">Link 1 </a>
       </li>
       <li>
           <a href="/" onclick="event.preventDefault()">Link 2</a>
       </li>
   </ul>
</body>
</html>
```

1

2

Dispatching custom events

นอกจากการกำหนดให้ Javascript จัดการ Events ที่เกิดขึ้นแล้ว **ยังสามารถสร้าง Events ขึ้นมาจาก Javascript ได้เช่นกัน เรียกว่า Dispatching events** โดยมีขั้นตอนดังนี้

1. Create Event Object
2. Dispatching Event

```
<html lang="en">
<body>
   <button id="elem" onclick="alert('Click!');">Autoclick</button>
   <script>
       let event = new Event("click");     // Event constructor
       elem.dispatchEvent(event);      // dispatchEvent
   </script>
</body>
</html>
```

Event constructor

เราสามารถสร้าง Events ขึ้นมาได้เองด้วย Javascript และกำหนดให้ Elements ต่างๆ ภายใน HTML ได้เหมือน Events ปกติ รวมถึงเรียกผ่าน Dispatching Event ได้เช่นกัน

```
<html lang="en">
<body>
   <h1 id="elem">Hello from the script!</h1>
   <script>
       // catch on document...
       document.addEventListener("hello", function (event) { // (1)
           alert("Hello from " + event.target.tagName); // Hello from H1
       });

       // ...dispatch on elem!
       let event = new Event("hello", { bubbles: true }); // (2)
       elem.dispatchEvent(event);
   </script>
</body>
</html>
```

Custom events

CustomEvent Object สามารถเพิ่ม property ของ Event ได้เพิ่มเติมได้

```
<html lang="en">
<body>
   <h1 id="elem">Hello for John!</h1>
   <script>
       // additional details come with the event to the handler
       elem.addEventListener("hello", function (event) {
           alert(event.detail.name);
       });

       elem.dispatchEvent(new CustomEvent("hello", {
           detail: { name: "John" }
       }));
   </script>
</body>
</html>
```

Forms

Navigation: form and elements

```
<html lang="en">
<body>
   <form name="my">
       <input name="one" value="1">
       <input name="two" value="2">
   </form>
   <script>
       // get the form
       let form = document.forms.my; // <form name="my"> element
       // get the element
       let elem = form.elements.one; // <input name="one"> element
       alert(elem.value); // 1
   </script>
</body>
</html>
```

Multiple elements with the same name

```
<html lang="en">
<body>
   <form>
       <input type="radio" name="age" value="10">
       <input type="radio" name="age" value="20">
   </form>
   <script>
       let form = document.forms[0];
       let ageElems = form.elements.age;
       alert(ageElems[0].value); // 10, the first input value
   </script>
</body>
</html>
```

Select and option

```
<html lang="en">
<body>
   <select id="select">
       <option value="apple">Apple</option>
       <option value="pear">Pear</option>
       <option value="banana">Banana</option>
   </select>
   <script>
       // all three lines do the same thing
       select.options[2].selected = true;
       select.selectedIndex = 2;
       select.value = 'banana';
   </script>
</body>
</html>
```

Event: submit

```
<html lang="en">
<body>
   <form onsubmit="return false">
       <input type="text" size="30" value="Focus here and press enter">
       <input type="submit" value="Submit" onclick="alert('click')">
   </form>
</body>
</html>
```

Method: submit

```
<html lang="en">
<body>
   <script>
       let form = document.createElement('form');
       form.action = 'https://google.com/search';
       form.method = 'GET';
       form.innerHTML = '<input name="q" value="test">';
       // the form must be in the document to submit it
       document.body.append(form);
       form.submit();
   </script>
</body>
</html>
```